

AORTA

Benchmark & Performance Report

Final benchmark characterization of the distributed systems project

System	Custom Layer-4 TCP load balancer + event-driven HTTP backends
Backend topology	3 backend servers: 8081 / 8082 / 8083
Data path	Redis-backed TaskManager + WorkerPool
Observability	Prometheus + Grafana
Benchmark client	wrk, 4 threads

Executive takeaway

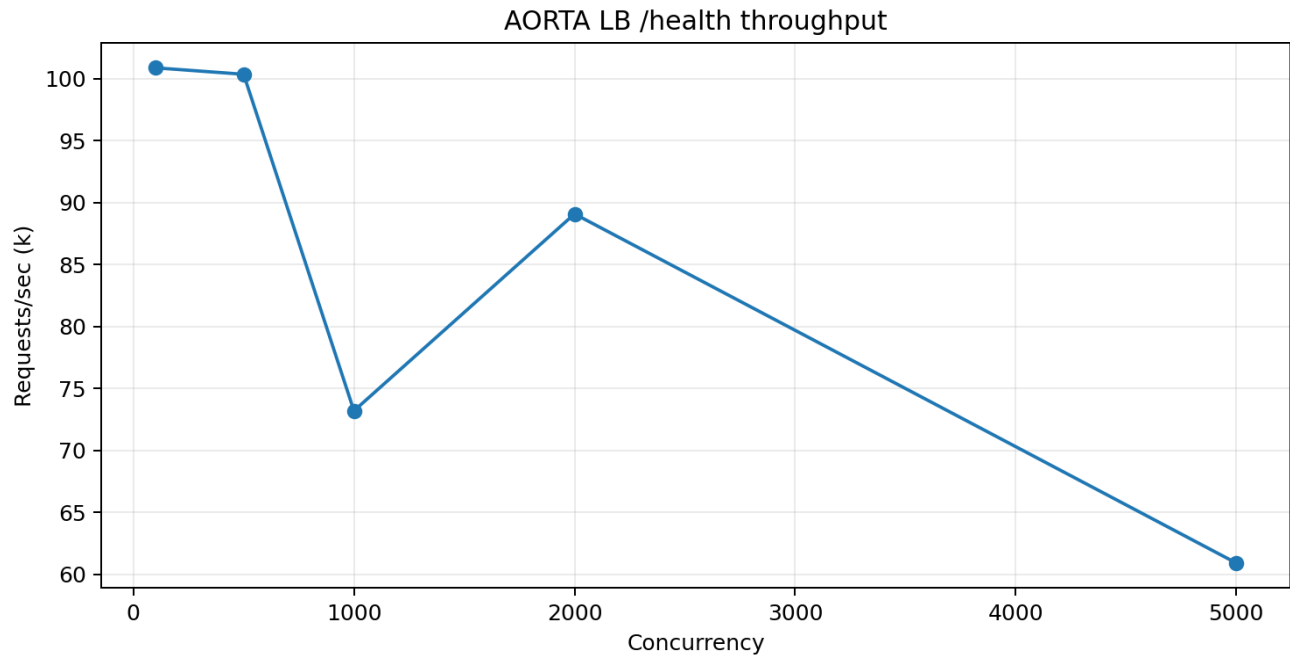
AORTA sustained roughly 100K requests/sec on the lightweight /health path at low-to-moderate concurrency. The Redis-backed /tasks path reached a measured peak of about 28.9K requests/sec in the optimized build, while direct backend measurement at c500 reached about 35.8K requests/sec. The benchmark data also shows the expected latency growth as concurrency rises.

Important measurement note: selected peak throughput figures and the latest latency-distribution run are not always from the same invocation. Where this occurs, the report keeps them explicitly separated rather than combining them into a fictitious single run.

1. Benchmark methodology

All latency-distribution measurements in the latest benchmark log were run with **wrk --latency -t4** for 30 seconds. The **/health** path was benchmarked at concurrency 100, 500, 1000, 2000 and 5000. The **/tasks** path was benchmarked at the same concurrency levels. A direct backend test was also run against server2 at c500 to estimate the load-balancer overhead.

Docker resource telemetry was deliberately excluded from the final report because the captured docker stats sample was not paired with a running wrk benchmark and therefore cannot support a meaningful CPU/throughput correlation.



2. /health results

Concurrency	RPS	Avg	p50	p99	Max
100	100,861.99	0.96 ms	0.85 ms	2.80 ms	16.29 ms
500	100,340.68	4.98 ms	4.33 ms	15.99 ms	76.39 ms
1,000	73,202.62	12.96 ms	11.86 ms	34.60 ms	145.95 ms
2,000	89,080.43	22.08 ms	19.98 ms	60.35 ms	360.08 ms
5,000	60,925.96	80.63 ms	74.94 ms	170.28 ms	602.31 ms

At c100, AORTA delivered 100,861.99 RPS with 0.85 ms p50 and 2.80 ms p99. At c500 it remained effectively flat at 100,340.68 RPS, while p99 increased to 15.99 ms. At c5000 throughput was 60,925.96 RPS and p99 reached 170.28 ms, showing the concurrency/latency trade-off clearly.

The c2000 result is non-monotonic in throughput relative to c1000 (89.08K vs 73.20K RPS). That variability is retained as measured; no smoothing or curve fitting has been applied.

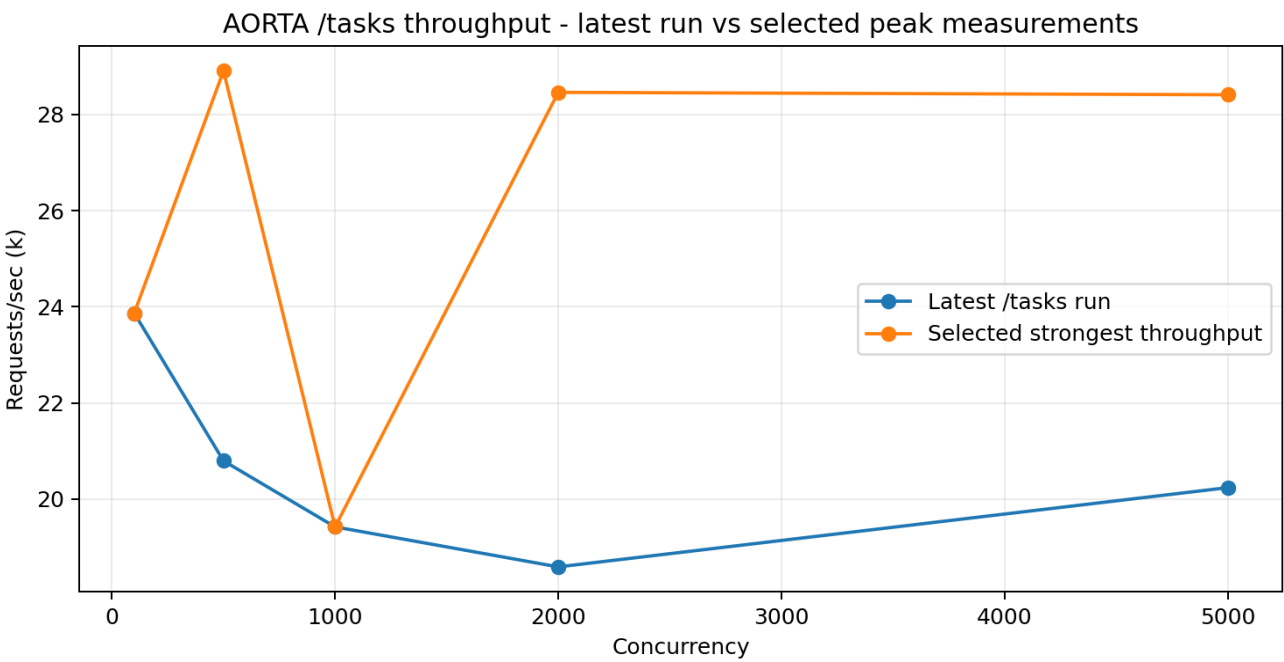
3. /tasks results

Latest latency characterization

Concurrency	RPS	Avg	p50	p99	Max
100	23,866.58	4.19 ms	3.84 ms	8.75 ms	20.28 ms
500	20,792.65	24.00 ms	22.12 ms	43.50 ms	79.09 ms
1,000	19,418.89	51.34 ms	46.77 ms	88.38 ms	148.75 ms
2,000	18,586.58	107.03 ms	102.07 ms	166.52 ms	281.72 ms
5,000	20,235.45	244.61 ms	222.51 ms	402.58 ms	691.84 ms

Strongest measured optimized-build throughput figures used for performance headline

Concurrency	Selected RPS	Source note
100	23,866.58	latest latency run
500	28,905.97	strongest measured optimized throughput
1,000	19,418.89	latest latency run
2,000	28,460.27	strongest measured optimized throughput
5,000	28,410.49	strongest measured optimized throughput



4. Direct backend vs load balancer

A direct server2 benchmark at c500 recorded 35,837.8 RPS with 13.82 ms average latency. The strongest optimized LB /tasks measurement at c500 recorded 28,905.97 RPS with 17.14 ms average latency from the earlier final optimization run.

Path	Concurrency	RPS	Avg latency
Direct server2 /tasks	500	35,837.8	13.82 ms
LB -> backend /tasks	500	28,905.97	17.14 ms

The LB path measured about 19.3% lower throughput than the direct backend path at c500. This is an observed end-to-end comparison, not a pure CPU cost measurement, because the two tests are separate benchmark invocations.

5. Application-side telemetry

Prometheus was queried after the benchmark series. The load balancer reported 3 total backends, 3 healthy backends, and zero backend failovers. Backend-side latency telemetry reported approximately 0.1 ms p50, 100 ms p95, 250 ms p99 and roughly 410-412 ms maximum at the captured snapshot.

Metric	Observed value
LB healthy backends	3
LB total backends	3
Backend failovers	0 for server1/server2/server3
Backend p50 latency	0.1 ms
Backend p95 latency	100 ms
Backend p99 latency	250 ms
Backend max latency	~410-412 ms

These application-side telemetry values should not be interpreted as identical to wrk latency. The AORTA metrics describe backend/application processing, while wrk measures client-observed end-to-end request latency through the LB.

The Prometheus snapshot also confirms that the benchmark environment remained in a 3/3 healthy-backend state with zero failovers during the captured observation.

6. Performance interpretation

High-throughput lightweight path

The /health endpoint demonstrates that the event-driven networking path can sustain around 100K RPS at c100-c500 before higher concurrency begins to increase latency materially.

Redis-backed task path

The /tasks path is substantially heavier because requests cross the asynchronous WorkerPool and Redis-backed TaskManager. The optimized implementation reached a measured peak near 28.9K RPS.

Concurrency trade-off

As concurrency rises, throughput eventually flattens while p50/p99 latency grows. At c5000, /tasks p99 was 402.58 ms in the latest latency run.

Load balancer overhead

At c500, the direct backend measurement was about 35.8K RPS versus 28.9K RPS through the LB, indicating meaningful proxy overhead while still preserving a large fraction of backend capacity.

Reliability

The captured Prometheus state showed 3/3 healthy backends and zero failovers, so the benchmark evidence does not indicate a reliability failure during the measured run.

7. Final benchmark summary

Peak /health throughput: 100,861.99 RPS at c100.

Peak optimized /tasks throughput observed: 28,905.97 RPS at c500.

Direct backend c500 throughput: 35,837.8 RPS.

Latest /tasks p99 at c5000: 402.58 ms.

Observed backend state: 3/3 healthy, zero failovers.

This report intentionally preserves the measured variation between benchmark runs. The headline values represent observed best results from the final optimized build, while the latest wrk --latency tables provide the cleanest latency characterization.